SE 423: Introduction to Mechatronics

Lecture 24: SLAM, Simultaneous Localization and Mapping

Marius Juston

Monday, April 20th, 2026

Review – 5-minute partner discussion

Talk to the person next to you and answer these questions.

- How far are they in their personal project?
  - What has their biggest challenge been? (other than just having the time to work on it)
- What is their plan for the final project?
  - If you are talking to someone in the same group, talk to another group!

Reminders

Office Hours:

- **Monday:** 4-6 PM**,** Neel
- **Tuesday:** 11-1 PM**,** Lakshmi
- **Tuesday:** 1-3 PM**,** Samuel
- **Tuesday:** 2-5 PM**,** Dan & Marius

Lab: Finishing Lab 7 this week. This a 2-week lab! This is the final lab! Afterwards you will just be working on your final project in your teams.

Homeworks:

- Remainder of homework 5 due on Gradescope on **April 21, 5PM**
- Next homework is primarily the **personal project** due on **April 22, 9AM**.

Questions?

Homework, Lab, LabVIEW?

SLAM

There are a couple of steps to understand SLAM, these will be split into 5 parts focused on 2D LiDAR SLAM (since that is the most important one for our robot):

- The robot turning on, in a completely unknown environment. Why is this hard?
- Localization, “where am I?” (ICP)
- Mapping, from the LiDAR, generate occupancy grids
- The back-end, pose graphs, factor graphs, and loop closure
- Real world situations, dynamic obstacles, ghost obstacles, robust SLAM

SLAM

There will be multiple components that will be used in the SLAM:

- Wheel odometry:
    - Fast, low-latency, but drifts
- 2D LiDAR
    - High-resolution geometric ground truth, but it is only one slice of the world
- The Front-End:
    - Converts the raw sequential scans into short-term motion estimates
- The Back-End:
    - Optimizes the entire history when loops close (loop closure)
- The Map:
    - A dynamic-aware model the planner can trust

Robot startup

SLAM

A mobile robot is powered on for the first time inside an unknown warehouse / location

What is unknown?

- No map
- No estimate of prior state (position)

Only has sensor streams! The LiDAR, IMU, Odometry

However, there are 2 questions that must be answered at every time-step to function properly and perform safe planning. What are they?

- Where am I right now? (Localization)
- What does the world around me look like (Mapping)

SLAM

For localization, why not just use GPS?

- GPS needs line-of-sight to 4+ orbiting satellites, buildings block this completely
- Civilian GPS accuracy is ~3-5 m, useless when aisles are 1.5 m wide
- Underground environments (mines, subways, basements) receive zero signal
- Indoor RTK and UWB beacons work, but require expensive pre-installed infrastructure

SLAM

For mapping why not just download a CAD model?

CAD drawings model the specific intended structure; however, not what the actual room is like today!

Furniture, people, tables, drift daily! Just think about this room!

A single misplaced chair renders a pre-programmed path to be unsafe (as you saw in lab adding an obstacle, we try to go point to point and even in the wall following sometimes it fails)

We need a map that reflects reality now!

When is it okay though to have a predefined map?

SLAM

SLAM, which stands for Simultaneous Localization and Mapping, tries to solve both at the same time.

There is a paradox though. Think about the chicken-and-egg trap…

- To build an accurate map you need to know your exact location at every scan
- To know your exact location, you need an already-existing map to compare against!

Classical robotics solves this separately; SLAM does both at the same time!

Every millisecond:
- Estimate motion
- Predict where the world should be
- Refine both!

SLAM

Essentially SLAM is trying to maximize the following probability:

$$\texttt{maximize} \quad P(x_{1:t}, m \mid z_{1:t}, u_{1:t})$$

Find the most likely trajectory x and map m given all observations z and controls u

In a single iteration, which do you think that the robot actually updates first? It's pose estimate or the map?

SLAM

Why do we need SLAM at all?

What some robotics examples where we want to map and plan trajectories in unknown / updating environments?

- Robot vacuums must map the unknown living room the first time they're switched on
- Self-driving cars use SLAM to reconcile HD maps with new construction, detours, and snow
- Mars rovers have no GPS and no human cartographer - everything is SLAM by necessity!
- Search-and-rescue drones enter collapsed buildings no person has ever mapped before

SLAM

What makes a good SLAM system?

- Real-time: the pose estimate must arrive before the robot crashes into the wall
- Globally consistent: no double walls, bent corridors, or duplicate rooms in the final map
- Robust to noise, to moving people, to sensor dropouts, to wheel slip on tile floors
- Memory-bounded: a $50 embedded CPU must hold a 10,000 m^2 facility's map
- Lifelong: the map must stay valid for weeks as the environment slowly evolves

Which of these five requirements do you think was the hardest to solve historically? Why?

SLAM

Global Consistency is the hardest to solve.

In early SLAM, robots relied primarily on Dead Reckoning or incremental scan-matching. Because every sensor measurement has a non-zero covariance, these small errors integrate over time.

The hardest part of global consistency is **Loop Closure**. Imagine a robot traveling a 1-kilometer loop. Even a tiny error of 0.5° in orientation can result in the robot returning to its starting point and perceiving the wall to be several meters away from where it "should" be.

The system must recognize a previously visited location using only sensor data (often called Place Recognition or Appearance-based SLAM).

SLAM

- The pose that the SLAM is optimizing for,

$$X_t = [x_t \quad y_t \quad \theta_t]$$

$$T_t = \begin{bmatrix} \cos\theta & -\sin\theta & x \\ \sin\theta & \cos\theta & y \\ 0 & 0 & 1 \end{bmatrix}$$

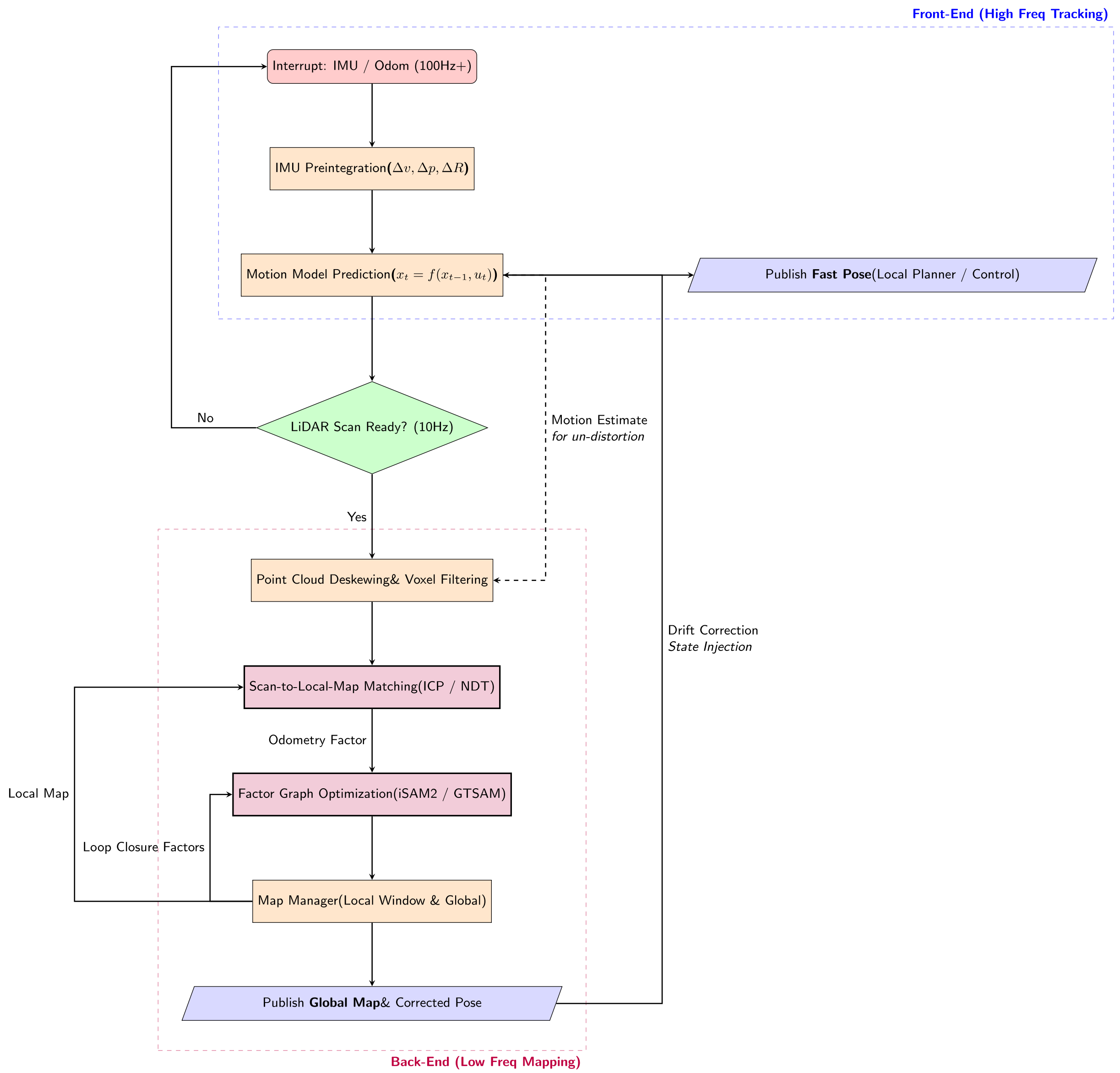
Every SLAM algorithm optimizes some collection of these pose triples over time. Since again we are optimizing the robot's position / trajectory that it has taken.

SLAM

The SLAM Loop:

- Read odometry: produce fast prior estimate of new pose
- Read LiDAR: capture current snapshot of the world
- Scan-match the snapshot again recent scans: refine the pose estimate
- Fuse LiDAR correction with odometry prior: publish the best pose
- Updating the map using the newly confirmed pose: repeat

SLAM



Front-End (High Freq Tracking)
Interrupt: IMU / Odom (100Hz+)
IMU Preintegration($\Delta v, \Delta p, \Delta R$)
Motion Model Prediction($x_t = f(x_{t-1}, u_t)$)
Publish **Fast Pose**(Local Planner / Control)
No
LiDAR Scan Ready? (10Hz)
Motion Estimate
for un-distortion
Yes
Point Cloud Deskewing& Voxel Filtering
Drift Correction
State Injection
Scan-to-Local-Map Matching(ICP / NDT)
Odometry Factor
Local Map
Factor Graph Optimization(iSAM2 / GTSAM)
Loop Closure Factors
Map Manager(Local Window & Global)
Publish **Global Map**& Corrected Pose
Back-End (Low Freq Mapping)

Where am I?

SLAM

We need to perform localization, specifically with LiDAR, we are aligning, the newly provided scan with the previous scan centimeters at a time.

Every scan-match boils down to finding one rigid transformation

- Given a source point cloud P (current scan) and a target Q (previous scan or map)
- Find the rotation R and translation t that best overlays P onto Q
- "Best" = minimizes a chosen error metric over all matched point pairs

If we know how the world moved relative to the robot, we know how the robot moved

$$T^* = (R^*, t^*) = \underset{\mathrm{R,t}}{\operatorname{argmin}} \sum_i error(Rp_i + t, q_i)$$

- The 1D wall example,

Imagine your robot is in a 1D hallway facing a wall,

Time step 1: The target Q,
- The robot is at global position 0
- The wall is 5 meters ahead
- The LiDAR records a point at $\mathrm{q} = 5$. This is our prior scan

Time step 2: The Source P,
- The robot drives forward 2 meters.
- The wall has not moved, but because the robot is closer the LiDAR measures the wall as being only 3 meters away
- The LiDAR records a new point at $\mathrm{p} = 3$, this is the new scan

If we just blindly drop the new point P and the old point Q into the same local coordinate space without moving them, they do not align!

The algorithm sees a point at 3 and a point at 5!

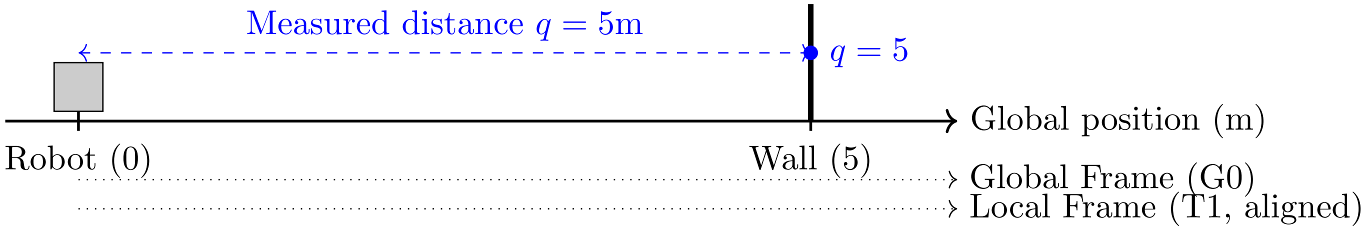
Now we apply our objective function to find the transformation t (ignoring rotation R for this 1D example):

$$t^* = \underset{t}{\operatorname{argmin}} \sum (p + t - q)^2$$

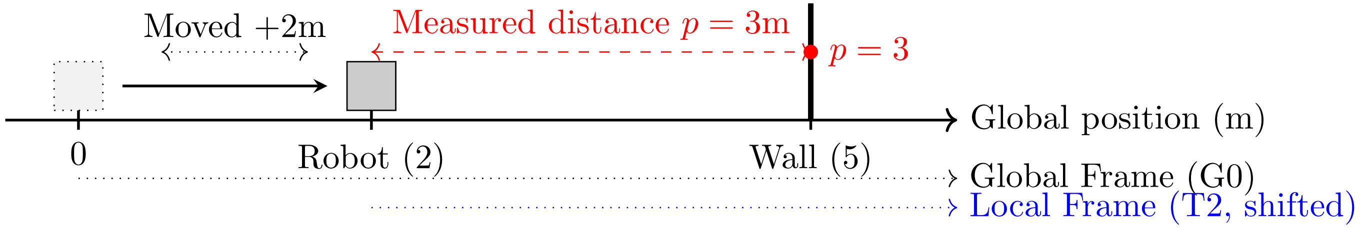
The algorithm thus asks: “How much do I need to shift the new point ($\mathrm{p} = 3$), so it sits as best it can on top of the old point ($\mathrm{q} = 5$)”

What is the optimal t in this case?

Step 1: Initial state and measurement



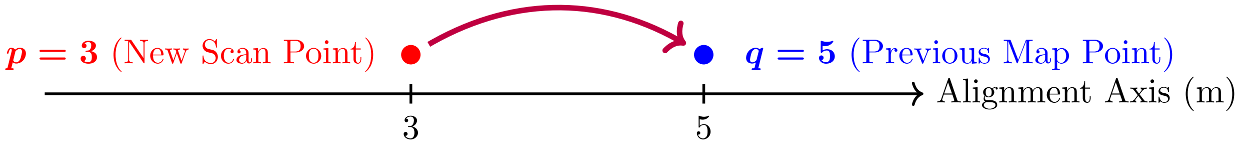
Step 2: Moved +2m, remeasured wall



Step 3: Conceptual alignment finding t

Find t to align: $3 + t = 5 \Rightarrow t = 2$

$\boldsymbol{t} = \mathbf{2}$ (robot movement)



SLAM

- The algorithm tells you that to align the scans, you must apply a translation of +2 meters to the new scan.

That +2 meters is exactly the distance the robot drove forward!

The matches are indeed close to each other in the physical world, but they are offset in the sensors by the exact inverse of the robot's motion.

By finding the transformation (R, t) that shifts the new local scan (P) to overlay perfectly onto the global map (Q), you are simultaneously calculating the robot's current pose in the world!

Three Families of Localization Methods

They differ in how many hypotheses (position estimates) they track and how they correct errors

1 Scan Matching (ICP)

- Maintains ONE best guess.
- Iteratively refines it by aligning the latest LiDAR scan to the map.
- Fast, deterministic, fragile at initialization.

2 Particle Filter (MCL)

- Tracks HUNDREDS of weighted hypotheses in parallel.
- Probabilistic, handles kidnapping, but memory-heavy for large maps.

3 Graph Optimization

- Maintains a graph of every past pose + constraints.
- Solves them all simultaneously.
- Today's gold standard for global consistency.

- Modern SLAM combine all three: ICP for the front-end, filters for relocalization, graphs for the backend

ICP

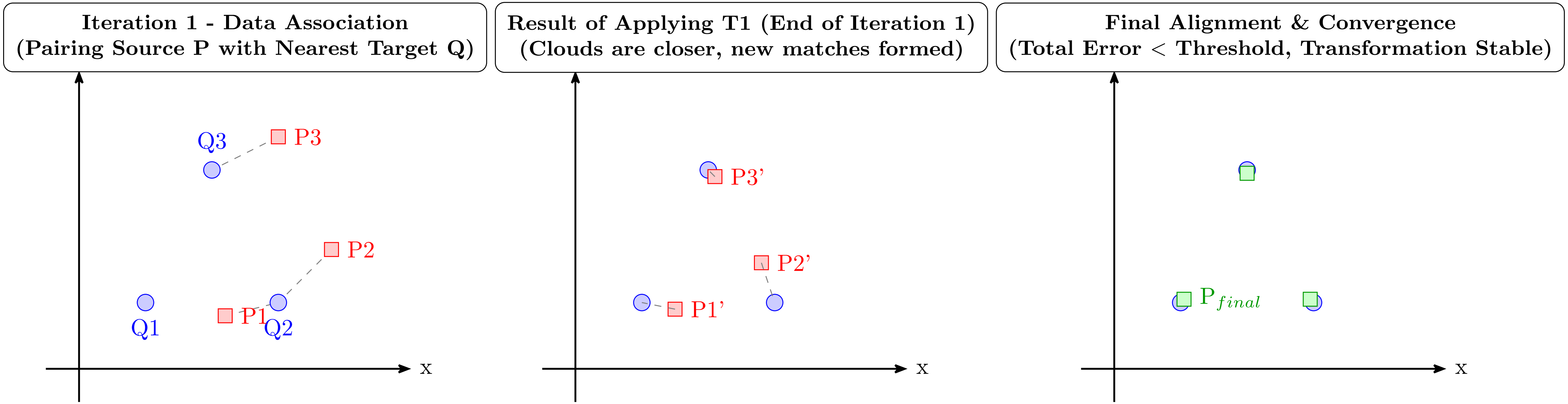
ICP

ICP stands for Iterative Closest Point:

Repeat the following 4 steps until the transformation stops changing:

- Data Association: pair each source point with its nearest target point
- Transformation: solve for the R, t that best aligns the matched pairs
- Apply: move the source cloud by that transformation
- Check: if the error has dropped below a threshold stop. Otherwise loop.

ICP



Iteration 1 - Data Association
(Pairing Source P with Nearest Target Q)
Q3
P3
P2
Q1
P1
Q2
x
Result of Applying T1 (End of Iteration 1)
(Clouds are closer, new matches formed)
P3'
P2'
P1'
x
Final Alignment & Convergence
(Total Error < Threshold, Transformation Stable)
P_{final}
x

Variant 1: Point-to-Point ICP ("Vanilla")

The starting point of the ICP methods,

- Match each source point p_i to its geometrically nearest target point q_i
- Measure the straight-line Euclidean distance between each matched pair
- Sum the squared distances and minimize over R, t

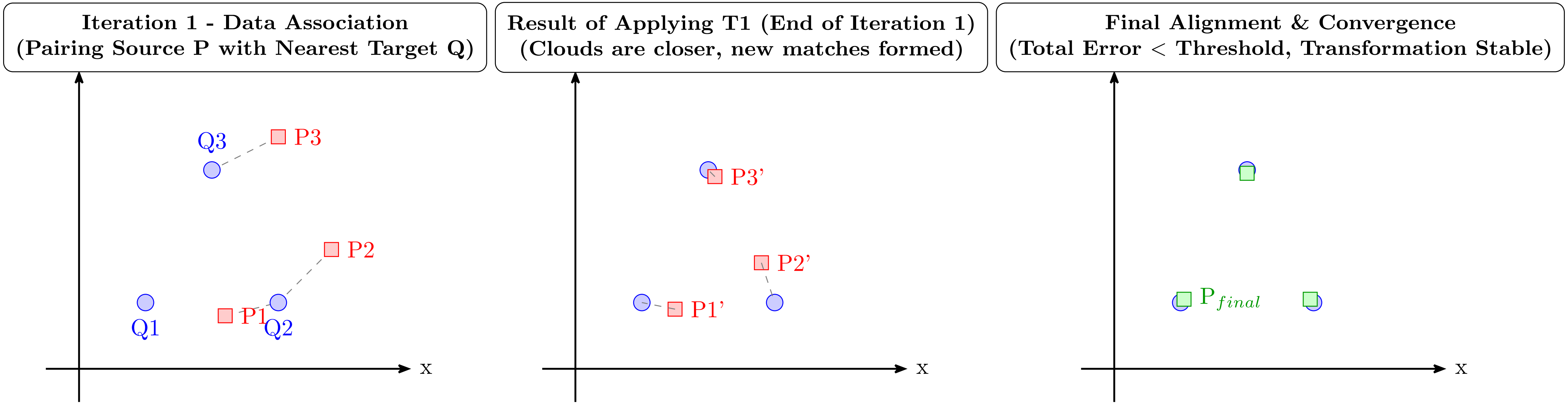
This is exactly what is illustrated in the figure.

What happens if you start the initial guess very wrong, say 90 degree off?

Works very well when two-point clouds start near each other and have clear features.

Falls apart in featureless or repetitive environments (more on that soon)

ICP



Iteration 1 - Data Association
(Pairing Source P with Nearest Target Q)
Q3
P3
P2
Q1
P1
Q2
x
Result of Applying T1 (End of Iteration 1)
(Clouds are closer, new matches formed)
P3'
P2'
P1'
x
Final Alignment & Convergence
(Total Error < Threshold, Transformation Stable)
P_{final}
x

Variant 1: Point-to-Point ICP ("Vanilla")

- For every matched pair (p_i, q_i), the residual is the 3D (or 2D) vector from p_i to the transformed q_i

The objective function is the sum of the squared residual magnitudes, a least-squares problem!

Given that this is a least-squares problem what is the advantage?

There is a **closed-form** solution via Singular Value Decomposition (SVD)

No iterative non-linear solver needed for each inner step!

$$E(R,t) = \sum_i ||R \cdot p_i + t - q_i||^2$$

Variant 1: Point-to-Point ICP ("Vanilla")

The P2P ICP can be solved in 4 steps:

1. Compute the centroids $\bar{p}$ and $\bar{q}$ of both point clouds
2. Build the 2×2 cross-covariance matrix $H = \sum_i (p_i - \bar{p})(q_i - \bar{q})^\top$
3. Decompose $H = U \cdot \Sigma \cdot V^\top$ using Singular Value Decomposition
4. Rotation: $R = V \cdot U^\top$ Translation: $t = \bar{q} - R \cdot \bar{p}$
5. Done!

Variant 1: Point-to-Point ICP ("Vanilla")

- $$E(R,t) = \sum_i ||R \cdot p_i + t - q_i||^2$$

Why is this equation quadratic (and therefore solvable exactly) despite containing rotations?

Rotations involve non-linear trigonometry (sines/cosines), which usually require iterative guessing. How do we solve for it exactly in one step?

After shifting both point clouds to their center of mass, the translation t becomes zero.

$$E(R) = \sum_i ||R \cdot p_i' - q_i'||^2$$

Variant 1: Point-to-Point ICP ("Vanilla")

- We must find the Rotation matrix R that minimizes the Sum of Squared Errors (SSE) between the centered source points (p_i') and target points (q_i').

$$E(R) = \sum_{i=1}^{N} || Rp_i' - q_i' ||^2$$

The geometric properties of rotation matrices cause the non-linear parts of the equation to self-destruct.

Variant 1: Point-to-Point ICP (“Vanilla”)

Expand the Quadratic: Since $||v||^2 = v^T v$, we expand the error function:

$$E(R) = \sum_{i=1}^{N} \left((p_i')^T R^T R p_i' - 2(q_i')^T R p_i' + (q_i')^T q_i' \right)$$

The Cancellation: R is a valid rotation matrix, meaning it is orthonormal ($R^T R = I$). Rotating a vector does not change its length.

Applying the Property:

$$(p_i')^T R^T R p_i' \Rightarrow (p_i')^T I p_i' \Rightarrow ||p_i'||^2$$

The Result: The quadratic R^2 term completely vanishes, leaving only a linear relationship!

Variant 1: Point-to-Point ICP ("Vanilla")

-

$$E(R) = \sum_{i=1}^{N} \left((p_i')^T R^T R p_i' - 2(q_i')^T R p_i' + (q_i')^T q_i' \right)$$

$$E(R) = \sum_{i=1}^{N} \left((p_i')^T p_i' - 2(q_i')^T R p_i' + (q_i')^T q_i' \right)$$

$$E(R) = \mathrm{c} - \sum_{i=1}^{N} (q_i')^T R p_i'$$

We can remove the constant terms from the error function!

Variant 1: Point-to-Point ICP ("Vanilla")

- $$E(R) = \mathrm{c} - \sum_{i=1}^{N} (q_i')^T R p_i'$$

With the **quadratic term** gone, the problem reduces to straightforward linear algebra.

Simplify: The equation is now just the static size of the point clouds (constants) minus a linear term.

To minimize the total error, we must maximize the negative term:

$$\text{Maximize: } \sum_{i=1}^{N} (q_i')^T R\, p_i'$$

Variant 1: Point-to-Point ICP ("Vanilla")

- Using a rule of linear algebra (the cyclic property of the Trace operator), we can rewrite the sum of these vector dot products as the Trace (sum of the diagonal) of a matrix multiplication:

$$\sum_{i=1}^{N} (q_i')^T R\, p_i' = \mathrm{Tr}\left(R \sum_{i=1}^{N} p_i' (q_i')^T \right)$$

The Trace Trick: We can rewrite this sum of vectors as the Trace of a matrix multiplication using the cross-covariance matrix (H):

$$\text{Maximize: } \mathrm{Tr}(R \cdot H)$$

Variant 1: Point-to-Point ICP (“Vanilla”)

The SVD Algorithm: Singular Value Decomposition is explicitly designed to maximize this matrix trace!

We decompose H and perfectly extract R:

$$H = U\Sigma V^T \Rightarrow R = VU^T$$

We can ignore Σ, because Σ represents scaling /stretching and we want our rotation matrix to maintain it’s validity.

Variant 1: Point-to-Point ICP (“Vanilla”)

The P2P ICP can be solved in 4 steps:

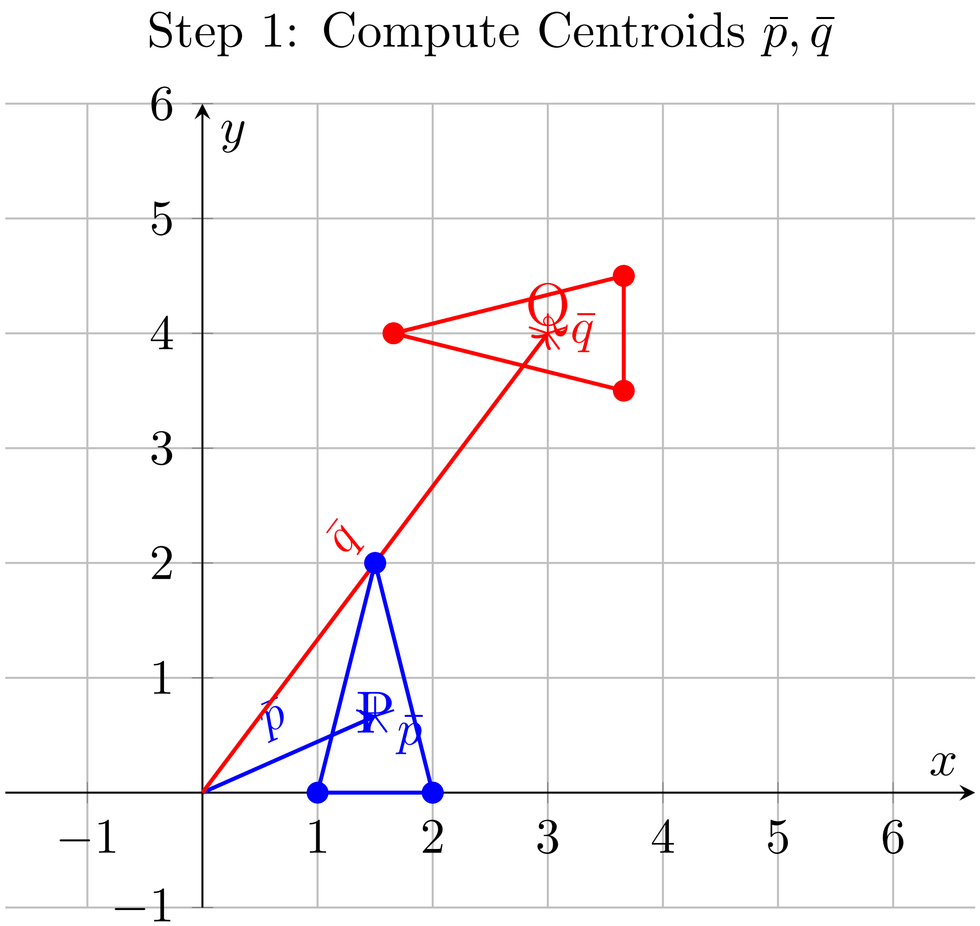
1. Compute the centroids $\bar{p}$ and $\bar{q}$ of both point clouds
2. Build the 2×2 cross-covariance matrix $H = \sum_i (p_i - \bar{p})(q_i - \bar{q})^\top$
3. Decompose $H = U \cdot \Sigma \cdot V^\top$ using Singular Value Decomposition
4. Rotation: $R = V \cdot U^\top$ Translation: $t = \bar{q} - R \cdot \bar{p}$
5. Done!

Variant 1: Point-to-Point ICP (“Vanilla”)

Very simple to implement in code as well!

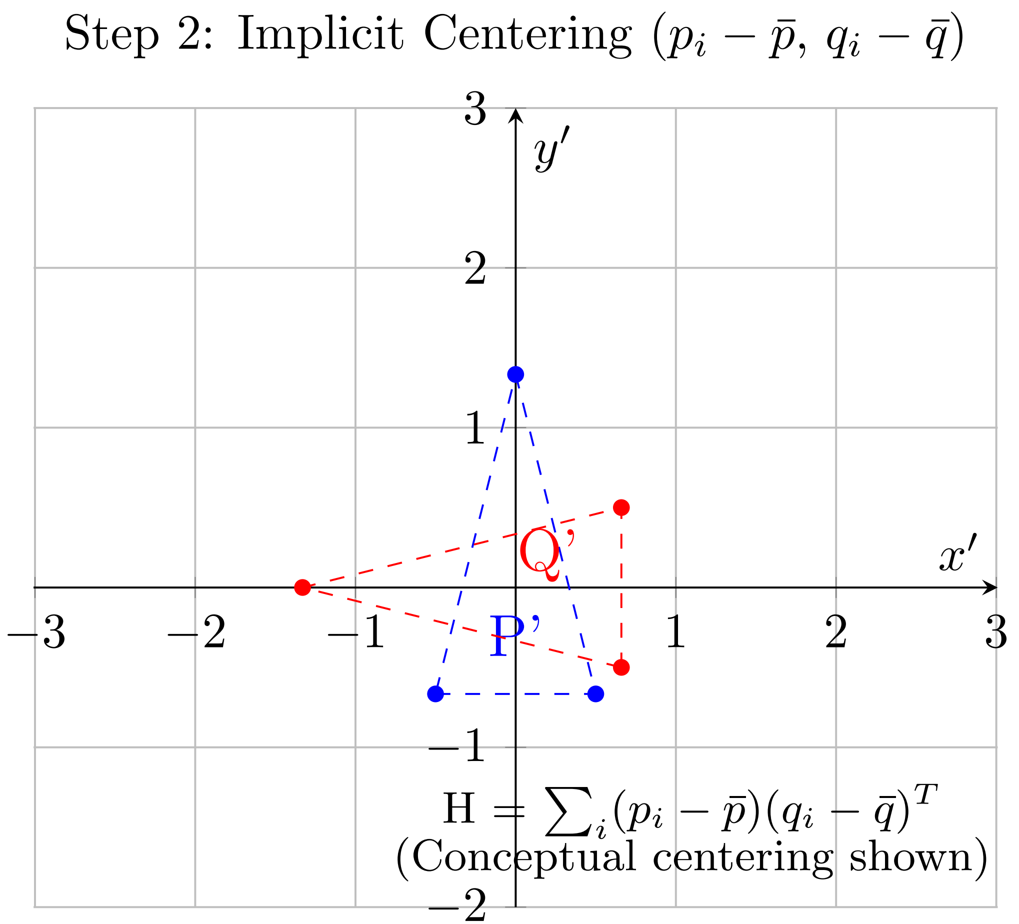
```
def p2p_icp(A, B):
    """
    Calculates the P2P ICP Closest Point Transformation
    Input:
        A: Nxm numpy array of source points
        B: Nxm numpy array of destination points
    Output:
        T: (m+1)x(m+1) homogeneous transformation matrix
    """
    m = A.shape[1]
    # --- Step 1: Compute the centroids ---
    centroid_A = np.mean(A, axis=0)
    centroid_B = np.mean(B, axis=0)
    # ------------------------------------

    # --- Step 2: Build cross-covariance matrix -
    AA = A - centroid_A
    BB = B - centroid_B
    H = np.dot(AA.T, BB)
    # -------------------------------------------

    # -- Step 3: SVD Decomposition ------
    U, S, Vt = np.linalg.svd(H)
    # -----------------------------------

    # --- Step 4: Extract R and t, from SVD ----
    R = np.dot(Vt.T, U.T)
    if np.linalg.det(R) < 0:
        Vt[m - 1, :] *= -1
        R = np.dot(Vt.T, U.T)
    t = centroid_B.reshape(-1, 1) - np.dot(R, centroid_A.reshape(-1, 1))
    #--------------------------------------------

    T = np.eye(m + 1)
    T[:m, :m] = R
    T[:m, -1] = t.ravel()
    return T
```

Variant 1: Point-to-Point ICP ("Vanilla")



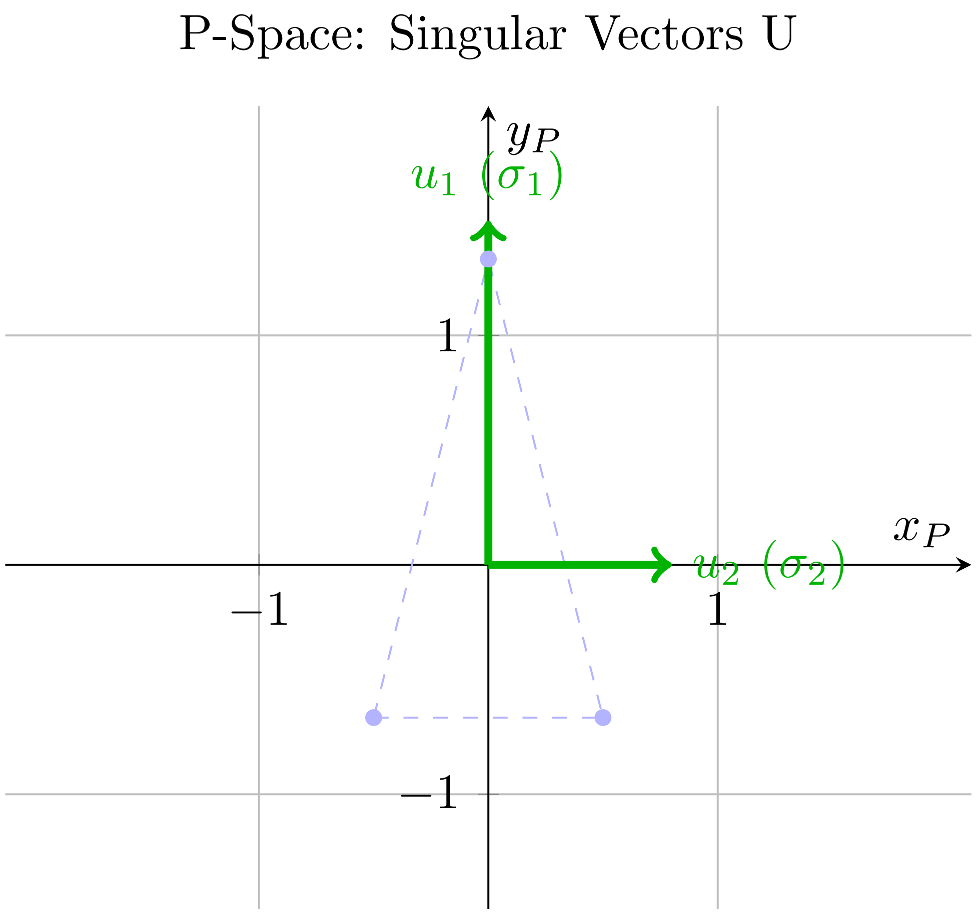
- Initial Point Clouds P (blue, offset triangle) and Q (red, offset and rotated triangle), with their calculated centroids ($\bar{p}$) and ($\bar{q}$) marked and vectors from the origin shown

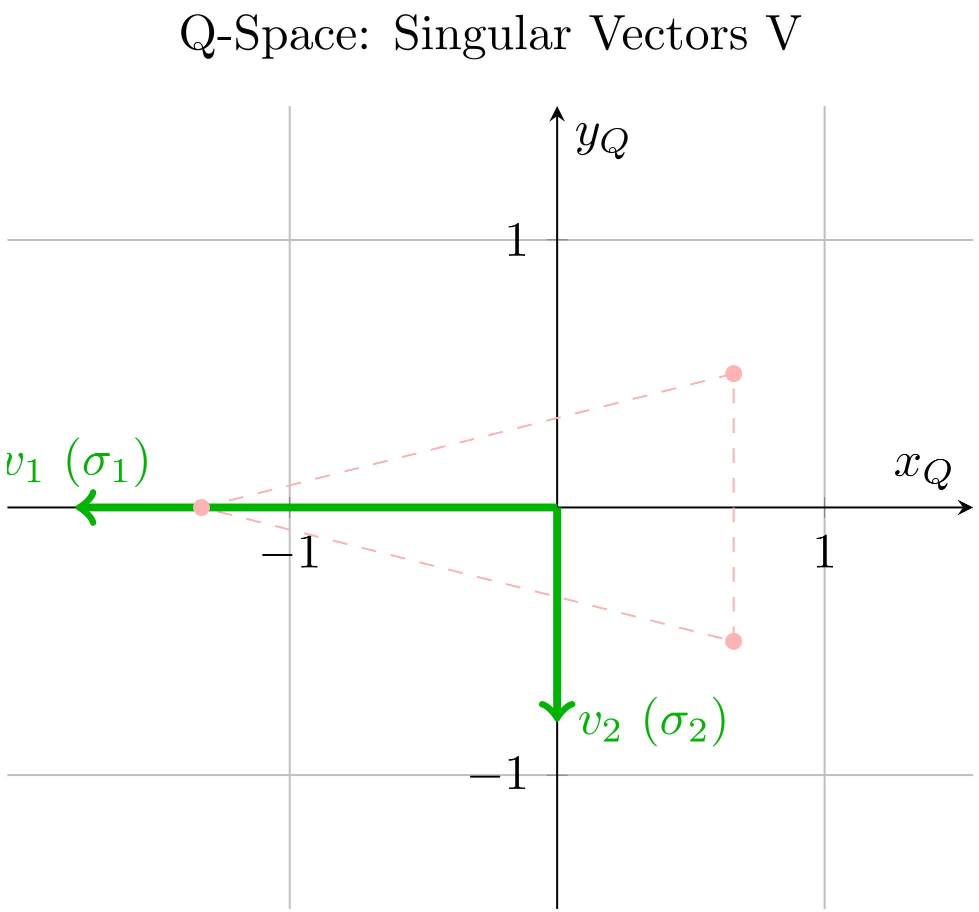
Variant 1: Point-to-Point ICP ("Vanilla")



- Visualization of Step 2 conceptually centering both point clouds at the global origin to compute the cross-covariance matrix H using the relative positions of points to their respective centroids

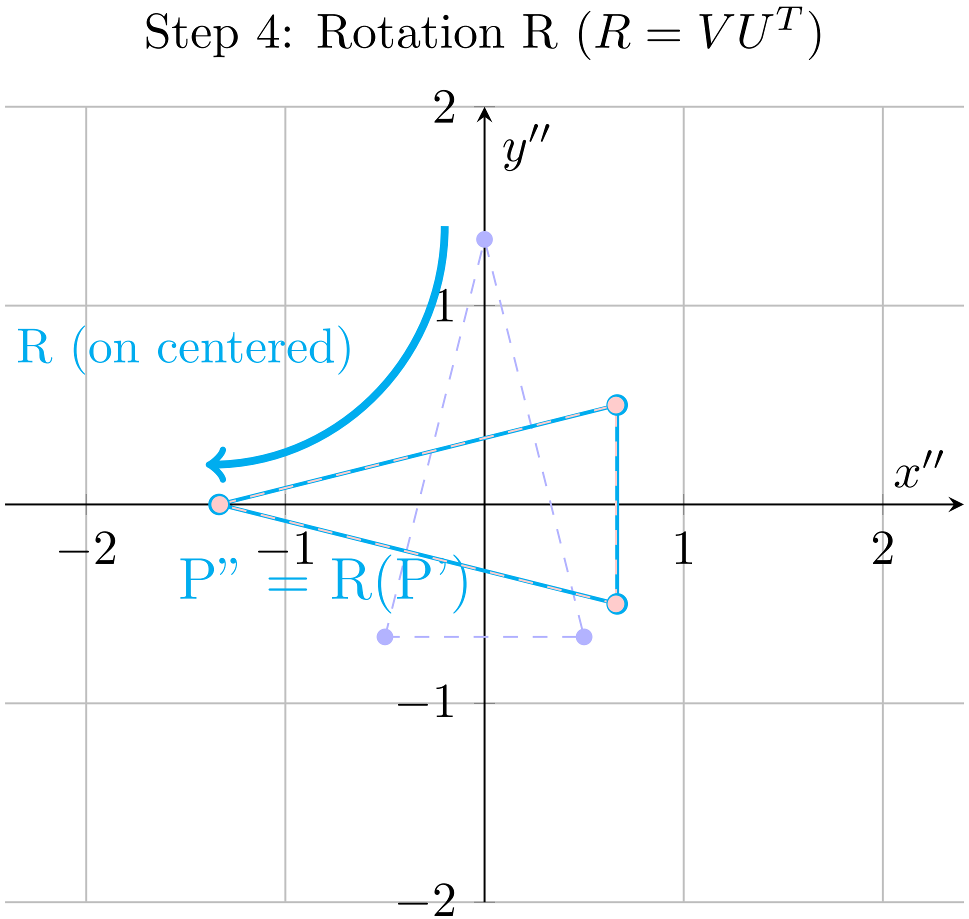
Variant 1: Point-to-Point ICP ("Vanilla")





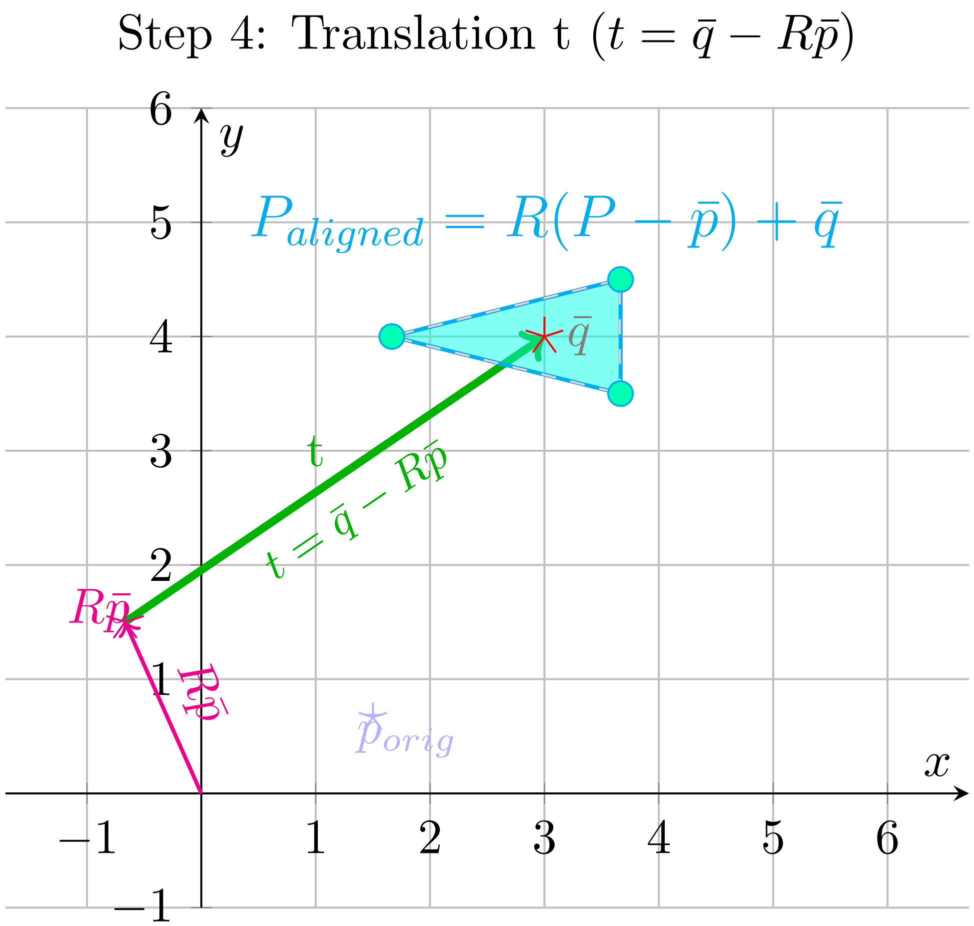
- Visualization of Step 3 SVD decomposition ($H = U\Sigma V^T$), showing the orthogonal singular vectors U and V as principal axes in the centered frames of reference for P and Q respectively, with illustrative singular values (σ_i) indicating the scale of variance along these directions.

Variant 1: Point-to-Point ICP ("Vanilla")



- Visualizing Step 4 rotation part, showing how the rotation matrix R is conceptually applied to the centered point cloud P' to align its principal axes with those of centered Q', resulting in the rotated, centered shape P''.

Variant 1: Point-to-Point ICP (“Vanilla”)



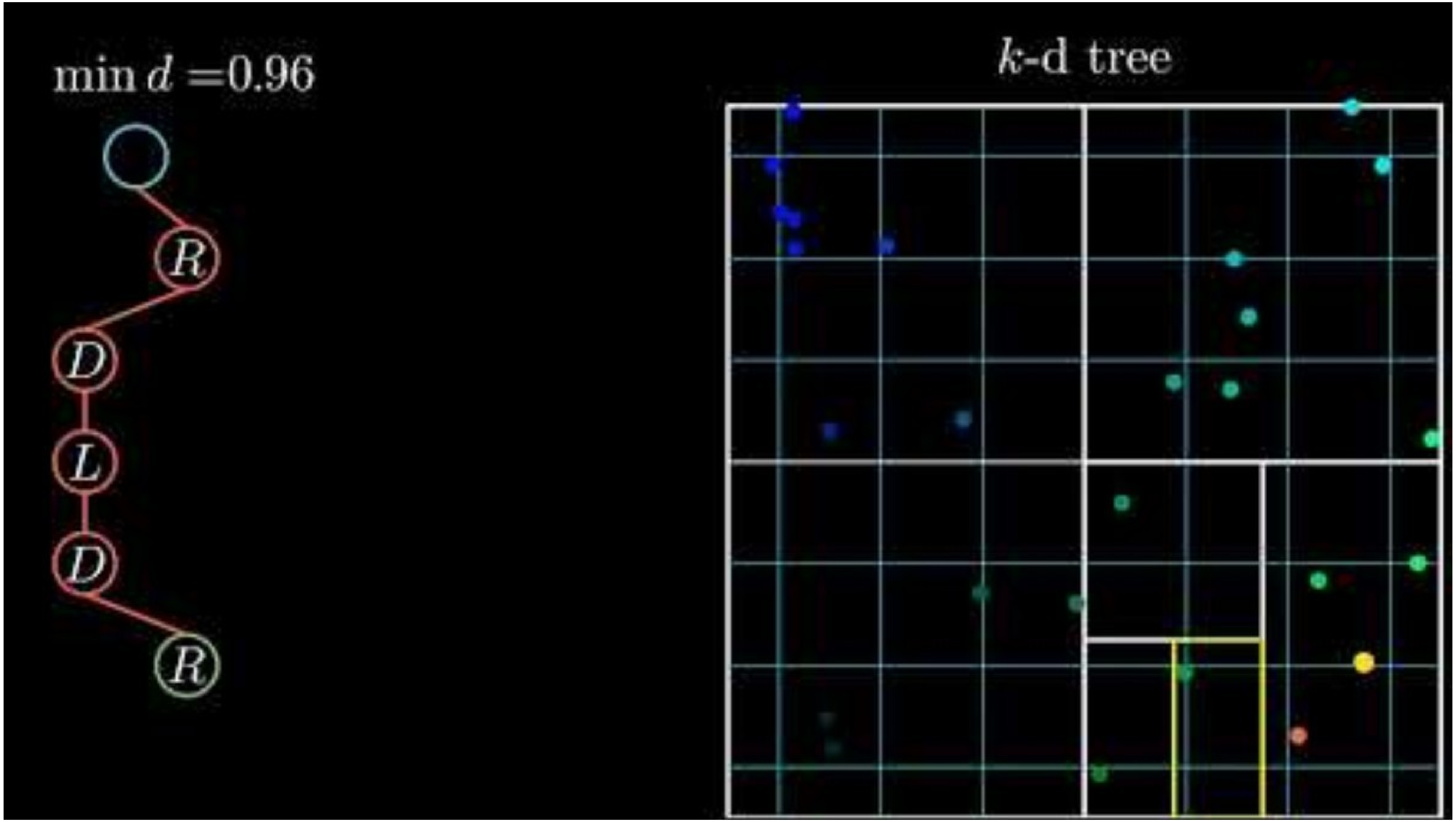
- Visualizing the Step 4 translation vector calculation $t = (\bar{q} - R\bar{p})$. The vector t shifts the rotated source centroid $(R\bar{p})$directly to the target centroid $(\bar{q})$. Conceptual perfect shape alignment (cyan filled triangle) is shown centered on $(\bar{q})$, aligning with the original Q shape.

Variant 1: Point-to-Point ICP ("Vanilla")

- The hidden costs the biggest problem is actually the first step!

- For N source points and M target points, naïve nearest-neighbor search is $O(\mathrm{N} \cdot \mathrm{M})$
  - Just check for each points between each point
- Solution:
  - Preprocess the target cloud into a **k-d tree**, binary space portioning structure $\mathrm{O}(\mathrm{M} \log \mathrm{M})$ construction time
  - K-d nearest-neighbor lookup drops to $O(log_2 M)$ per query! A major speedup!
  - Almost every ICP implementation in ROS, PCL and Open3D uses K-D trees internally!

Variant 1: Point-to-Point ICP ("Vanilla")



Variant 1: Point-to-Point ICP ("Vanilla")

Imagine a robot driving straight down a long, flat, featureless corridor!

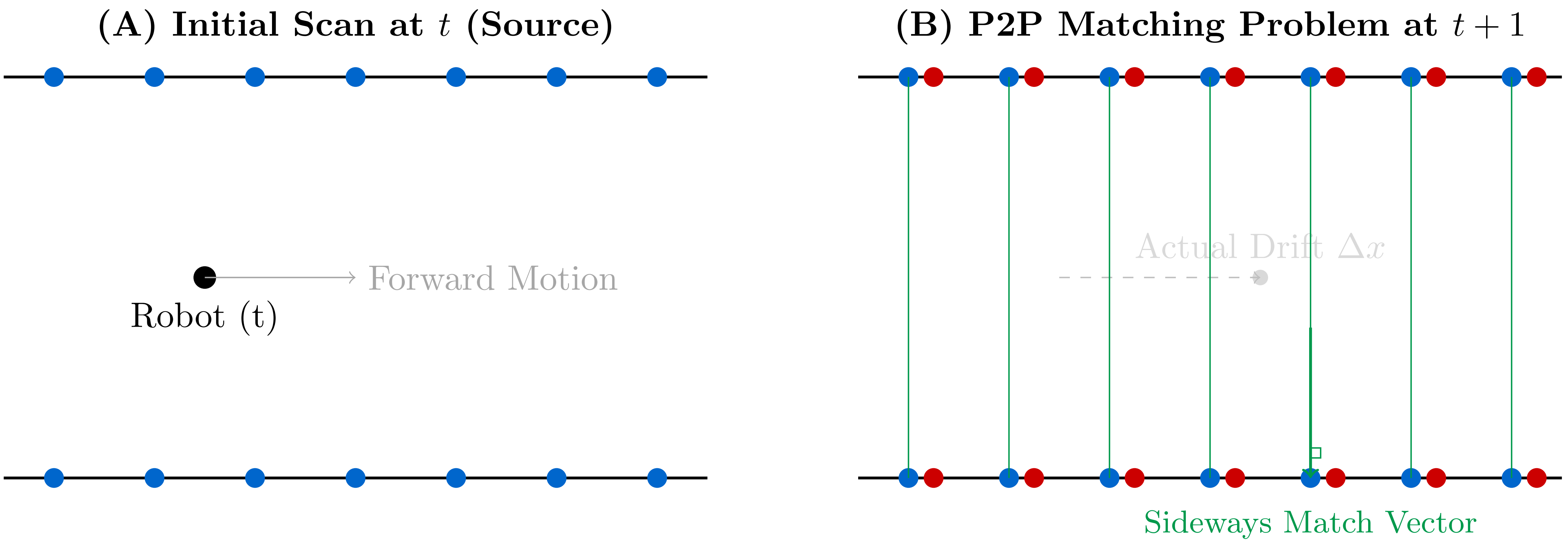
- The left and right walls look geometrically identical at timestep t and timestep $t + 1$
- P2P matches each source point to its nearest target point - but that's sideways, not forward!

What does the algorithm conclude?

'I haven't moved forward at all' and locks the map in place

The fundamental problem: P2P cannot distinguish 'slide along a wall' from 'did not move'

Variant 1: Point-to-Point ICP ("Vanilla")



Variant 2: Point-to-Line ICP (PL-ICP)

- The key insight is this, in 2D the world is mostly made of line segments (wall, doors, edges of boxes)

Instead of matching a source point to a single target point, match it to the nearest target **LINE**

For each target line segment, compute the unit normal n_i, (perpendicular to the near wall)

Only the component of the error **PERPENDICULAR** to the wall is penalized

A robot can now slide freely along a smooth corridor with the algorithm resisting it!

Variant 2: Point-to-Line ICP (PL-ICP)

- Residual = projection of $R \cdot p_i + t - q_i$ onto the surface normal n_i
- Motion parallel to the wall contributes ZERO error - the algorithm doesn't resist it, it just slides on the line

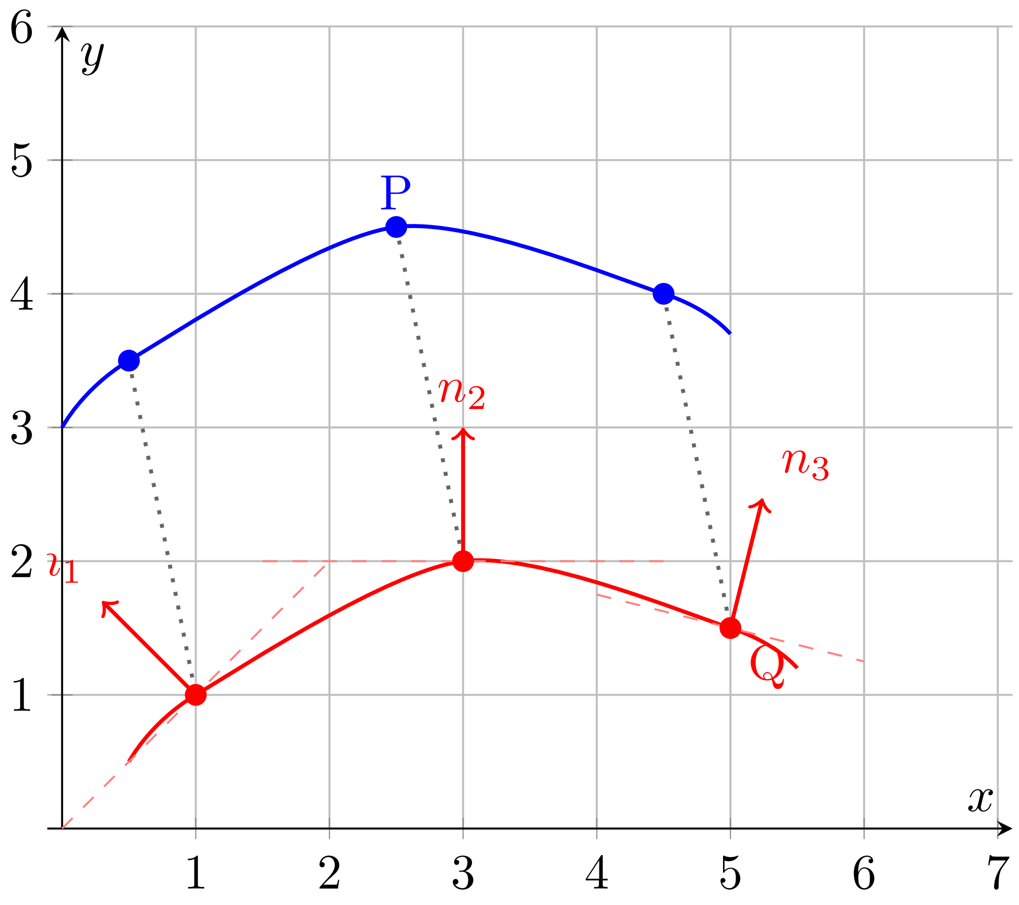
- For high-frequency LiDAR, the angle between consecutive scans is tiny, so we linearize, based on $\sin\theta = 1, \cos\theta = 1$
- This turns a hard non-linear problem into ordinary linear least squares!

$$E(R,t) = \sum_i \left|\left|(R \cdot p_i + t - q_i) \cdot n_i\right|\right|^2$$

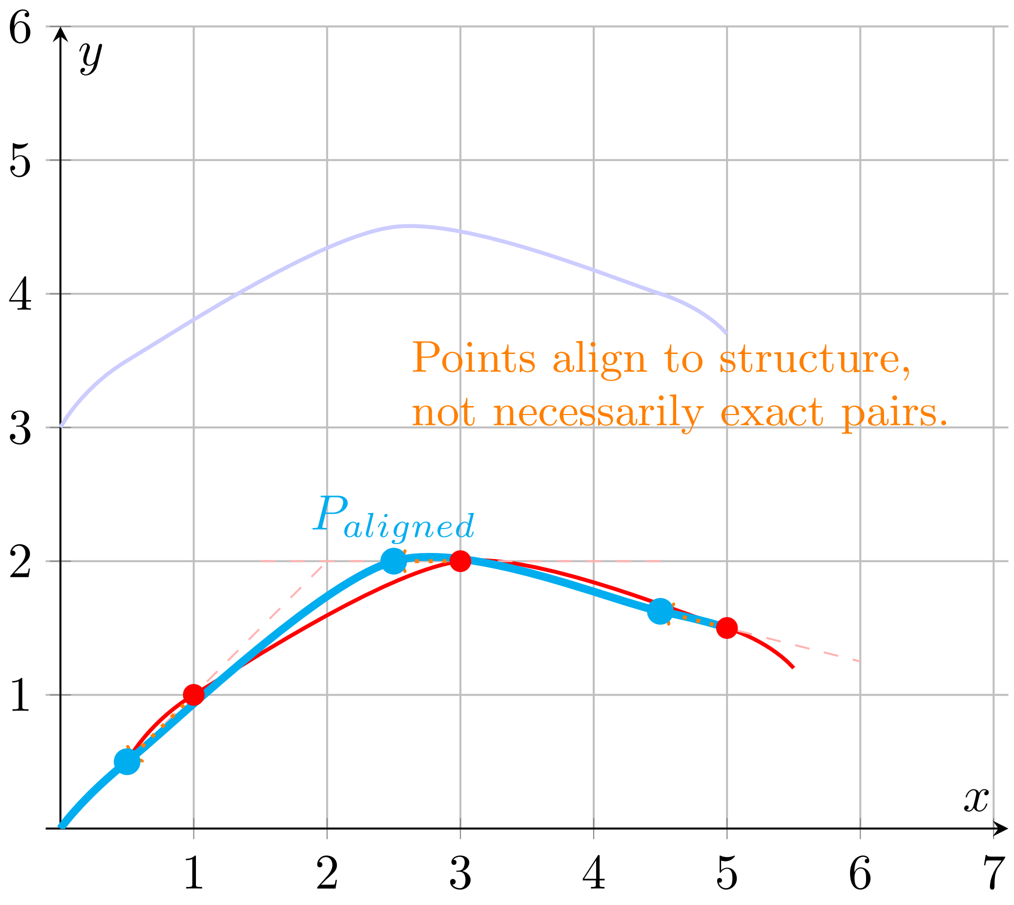
Parallel motion = zero error, $x \cdot n_i = 0$
Perpendicular motion = penalty, $x \cdot n_i = x$

Variant 2: Point-to-Line ICP (PL-ICP)

Step 1: Identify Correspondences & Target Normals (n_i)



Step 5: Apply Transform & Update (Iterate)



Variant 3: Point-to-Plane ICP

How do you think this generalizes in 3D?

- In 3D, a plane is the natural generalization of a line (floor, ceiling, wall surface)
- Every target point q_i gets a surface-normal vector n_i computed from its local neighbors
- Error is projected onto n_i exactly as in PL-ICP, motion along the plane is free
- Converges in far fewer iterations than P2P in structured indoor environments (given that 3D structures provide more constraints / unique presentations)

Variant 4: Generalized ICP (GICP)

What if BOTH clouds are noisy? We need uncertainty-aware matching.

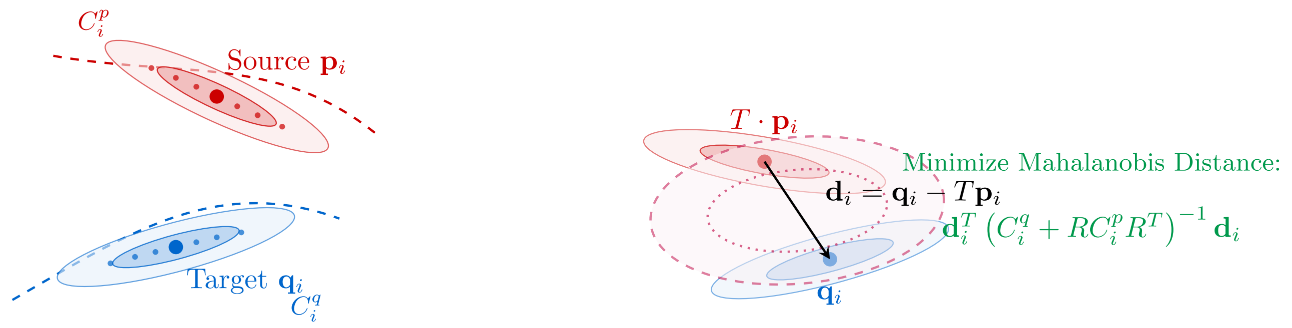
- P2P assumes source points are perfect, P2Plane assumes target surfaces are perfect
- Reality: both clouds carry uncertainty - laser range noise, quantization, timing jitter
- GICP attaches a 2×2 (or 3×3) covariance matrix to every point in both clouds
- A point on a flat wall has a 'pancake' covariance: certain perpendicular, uncertain along the wall
- Alignment is now done in probability space, not distance space

Variant 4: Generalized ICP (GICP)

- Both source and target have covariances C_p and C_q - we combine them into a weighting matrix
- The resulting form unifies P2P (isotropic noise) and P2Plane (degenerate planar noise)!
- One framework, tunable for the specific sensor and environment you have

$$E(R,t) = \sum_i d_i^{\top} \left(C_{q_i} + R \cdot C_{p_i} \cdot R^{\top} \right)^{-1} d_i$$

We minimize the “Mahalanobis distance” (just look it up, but it’s just the equation above).



(A) Probabilistic Surface Modeling
Each point is assigned a covariance matrix representing its local geometric neighborhood.

(B) Plane-to-Plane Alignment
Distance is weighted by the sum of covariances. Allows sliding along both source and target planes.

Variant 5: Normal Distributions Transform (NDT)

Throw away points, keep distributions!

ICP spends most of its time on k-d tree nearest-neighbor lookups
NDT eliminates this entirely, no point-to-point matching ever happens

- **Step 1**: divide the target map into a regular 2D grid of voxels
- **Step 2**: for each voxel, compute a Gaussian (mean μ, covariance Σ) over its resident points

Scan matching becomes find the transformation that MAXIMIZES the scan's probability under this field

Variant 5: Normal Distributions Transform (NDT)

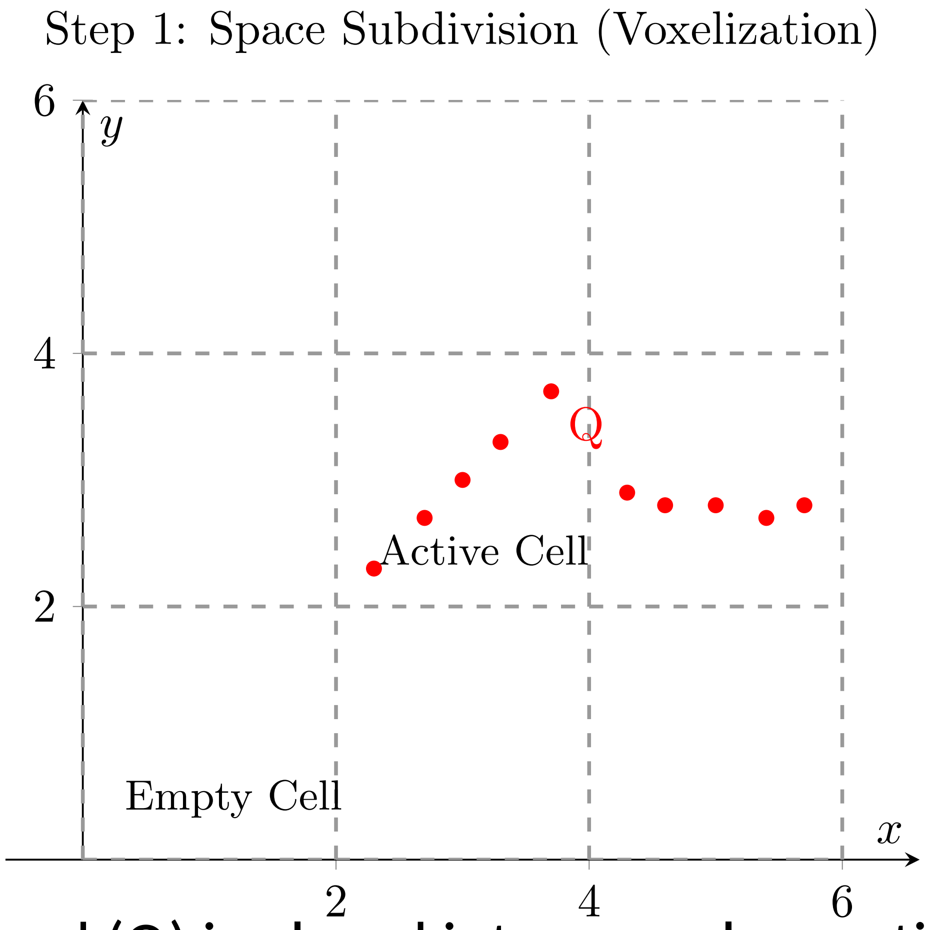
- For each source point p_i, evaluate the Gaussian of the cell it falls into
- Sum the log-probabilities (or negative probabilities) across all source points
- Use Newton's method to find the R, t that maximizes this score

Advantage: no neighbor search.
Disadvantage: heavily tuned by the voxel size you choose.

$$score(R,t) = -\Sigma_i e^{-\frac{1}{2}(R \cdot p_i + t - \mu_i)^\top \Sigma_i^{-1} (R \cdot p_i + t - \mu_i)}$$

Minimizing negative likelihood = maximizing the match probability

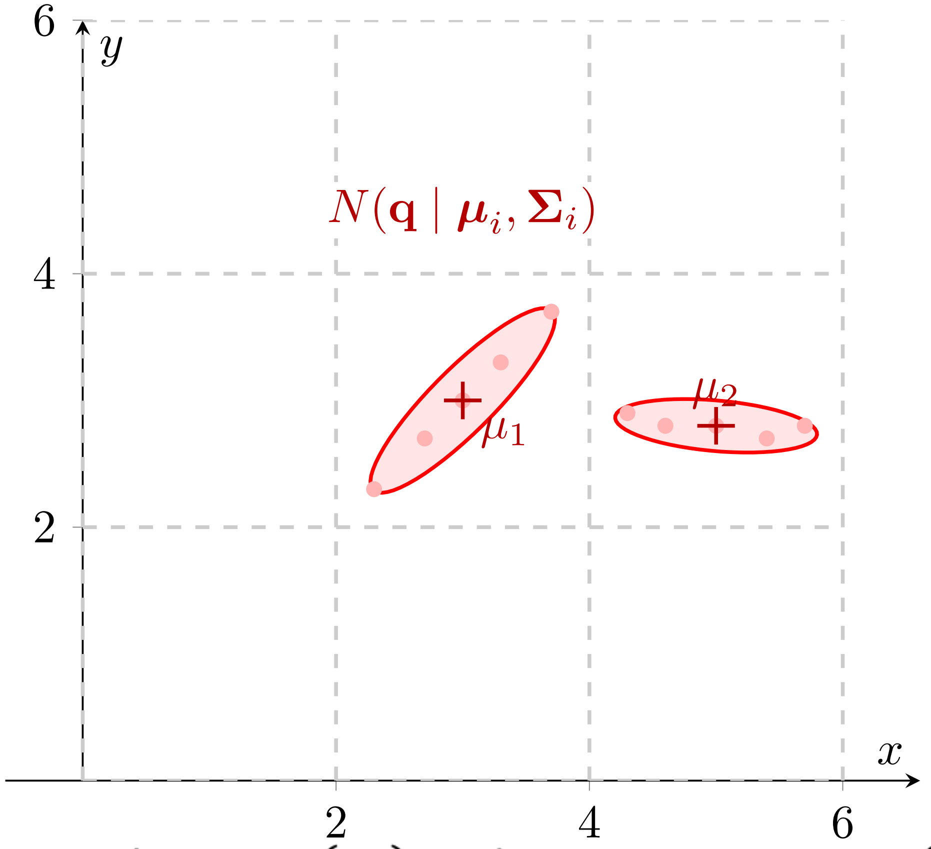
Variant 5: Normal Distributions Transform (NDT)



- The target point cloud (Q) is placed into a regular spatial grid (voxels in 3D, squares in 2D). Cells with fewer than a minimum number of points (e.g., 3 points) are usually ignored to prevent mathematical instability

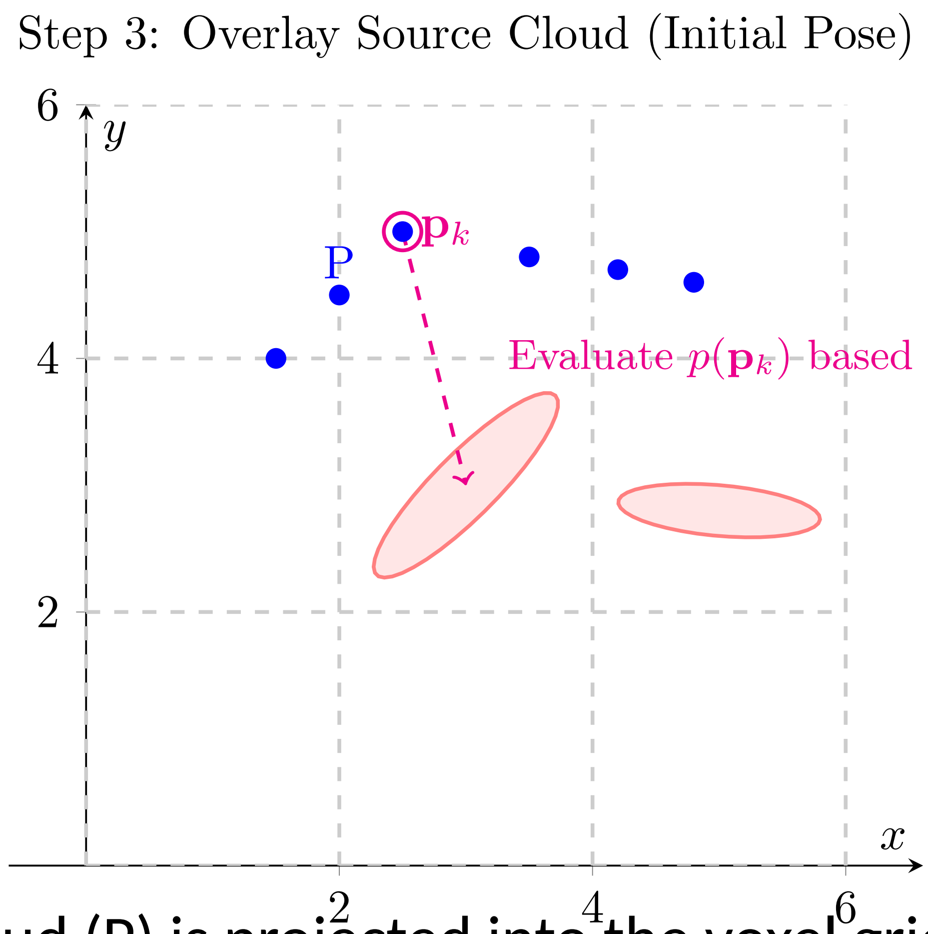
Variant 5: Normal Distributions Transform (NDT)

Step 2: Calculate Local Probability Density Functions (PDFs)



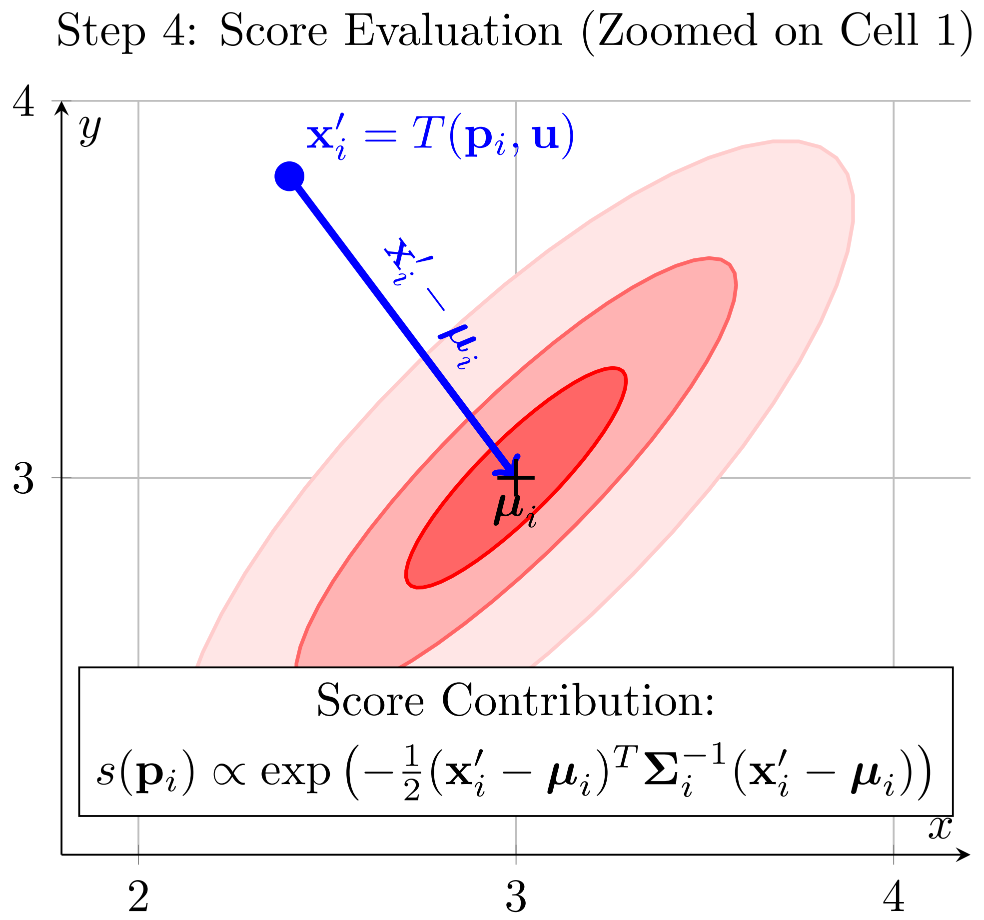
- For every active cell, the sample mean $(\boldsymbol{\mu_i})$ and covariance matrix $(\boldsymbol{\Sigma_i})$ of the target points are calculated. This transforms the discrete point cloud into a piecewise continuous set of multi-variate Gaussian probability distributions (represented here by covariance ellipses).

Variant 5: Normal Distributions Transform (NDT)



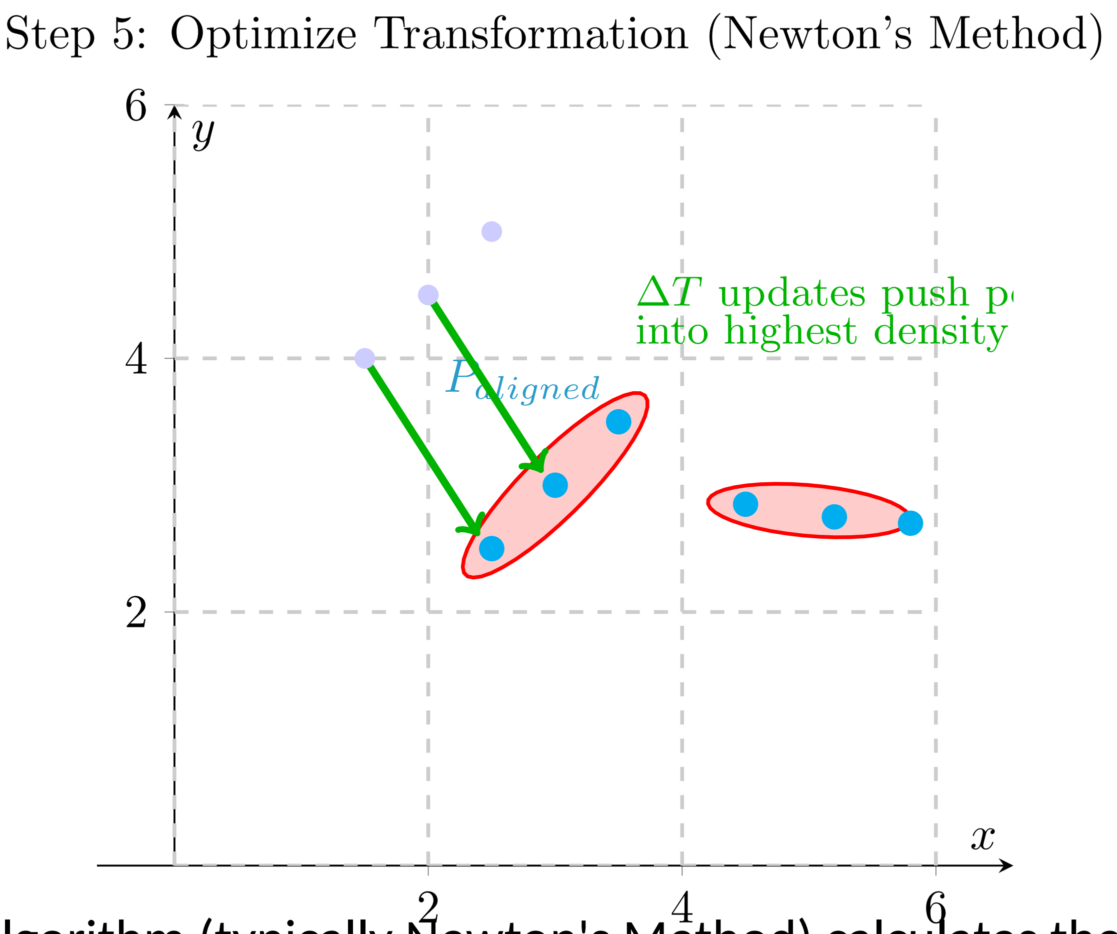
- The source point cloud (P) is projected into the voxel grid using its initial, unoptimized transformation. We determine which spatial cell each source point falls into (or its nearest neighbors if using smoothed NDT).

Variant 5: Normal Distributions Transform (NDT)



- NDT entirely drops discrete pairs. Instead, it evaluates the probability density of the target cell at the location of the transformed source point ($\mathbf{x}'_\mathbf{i}$). The objective function is to maximize the sum of these probabilities for all source points across the grid.

Variant 5: Normal Distributions Transform (NDT)



- An optimization algorithm (typically Newton's Method) calculates the Hessian and gradient of the objective function to update the transformation parameters (rotation and translation). The source cloud "falls" into the high-probability valleys defined by the target's Gaussian mixture.

NDT vs ICP - when to use which

- Cartographer (2D indoor) uses ICP-style. Autoware (3D outdoor) uses NDT. The sensor determines the algorithm.

ICP FAMILY

- Best for small point clouds (< 5,000 pts)
- Very accurate when features are strong
- Needs a good initial guess
- k-d tree dominates runtime
- Gold standard for 2D indoor SLAM
- Struggles with smooth, flat, featureless areas

NDT

- Best for dense 3D clouds (> 50,000 pts)
- Smooth cost landscape - larger basin of attraction
- Less sensitive to a bad initial guess
- No k-d tree needed - very fast
- Common in outdoor autonomous vehicles
- Accuracy capped by voxel resolution

ICP Family Failure Modes - What Still Breaks

Featureless environments (long smooth tunnels, empty parking lots)

- no geometric handle, huge amount of drift at the end, essentially just relies on odometry

Symmetric environments (hallway intersections that all look identical)

- perceptual aliasing, unknown rotation / translation

Large initial guess errors (>30°)

- any ICP will converge to a wrong local minimum

Dynamic environments with moving objects

Glass, mirrors, and retroreflectors

- The LiDAR returns the wrong values

Monte Carlo estimates

Questions?

The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN